

## PROJECT- ATM SIMULATION

```
import java.util.Scanner;

import java.util.HashMap;

import java.util.Map;


public class ATMSimulation {


    // User account data

    private static Map<String, UserAccount> accounts = new HashMap<>();

    private static UserAccount currentUser = null;

    private static Scanner scanner = new Scanner(System.in);


    public static void main(String[] args) {

        // Initialize with some dummy accounts

        initializeAccounts();


        System.out.println("===== WELCOME TO ATM SIMULATION =====");

        System.out.println();


        // Main program loop

        boolean running = true;

        while (running) {

            if (currentUser == null) {

                // Show login screen

                showLoginScreen();

            } else {

                // Show main menu for logged-in user

                running = showMainMenu();

            }

        }

    }

}
```

```
}  
}
```

```
System.out.println("\nThank you for using ATM Simulation. Goodbye!");  
scanner.close();  
}
```

```
private static void initializeAccounts() {  
    accounts.put("1234567890", new UserAccount("1234567890", "1111", "John Doe",  
1500.75));  
    accounts.put("9876543210", new UserAccount("9876543210", "2222", "Jane Smith",  
2500.00));  
    accounts.put("5555555555", new UserAccount("5555555555", "3333", "Robert  
Johnson", 500.50));  
    accounts.put("1111111111", new UserAccount("1111111111", "4444", "Emily Brown",  
10000.00));  
}
```

```
private static void showLoginScreen() {  
    System.out.println("----- LOGIN -----");  
    System.out.print("Enter Account Number: ");  
    String accountNumber = scanner.nextLine();  
  
    System.out.print("Enter PIN: ");  
    String pin = scanner.nextLine();  
  
    // Validate credentials  
    if (accounts.containsKey(accountNumber)) {  
        UserAccount account = accounts.get(accountNumber);
```

```

        if (account.validatePin(pin)) {
            currentUser = account;

            System.out.println("\n✅ Login successful! Welcome, " + account.getHolderName()
+ "!\\n");
        } else {
            System.out.println("\n❌ Invalid PIN. Please try again.\\n");
        }
    } else {
        System.out.println("\n❌ Account not found. Please try again.\\n");
    }
}

```

```

private static boolean showMainMenu() {
    System.out.println("==== MAIN MENU =====");
    System.out.println("1. Check Balance");
    System.out.println("2. Deposit Money");
    System.out.println("3. Withdraw Money");
    System.out.println("4. Transfer Money");
    System.out.println("5. Transaction History");
    System.out.println("6. Logout");
    System.out.println("7. Exit");
    System.out.print("Choose an option: ");

    int choice;

    try {
        choice = Integer.parseInt(scanner.nextLine());
    } catch (NumberFormatException e) {
        System.out.println("\n❌ Invalid input. Please enter a number.\\n");
    }
}

```

```
    return true;
}
```

```
switch (choice) {
    case 1:
        checkBalance();
        break;
    case 2:
        depositMoney();
        break;
    case 3:
        withdrawMoney();
        break;
    case 4:
        transferMoney();
        break;
    case 5:
        showTransactionHistory();
        break;
    case 6:
        logout();
        break;
    case 7:
        return false;
    default:
        System.out.println("\n ❌ Invalid option. Please try again.\n");
}
return true;
```

```
}
```

```
private static void checkBalance() {  
    System.out.println("\n----- BALANCE INQUIRY -----");  
    System.out.printf("Current Balance: $%.2f\n", currentUser.getBalance());  
    System.out.println("-----\n");  
}
```

```
private static void depositMoney() {  
    System.out.println("\n----- DEPOSIT -----");  
    System.out.print("Enter amount to deposit: $");  
  
    try {  
        double amount = Double.parseDouble(scanner.nextLine());  
        if (amount <= 0) {  
            System.out.println(" ❌ Amount must be greater than zero.\n");  
            return;  
        }  
  
        currentUser.deposit(amount);  
        System.out.printf(" ✅ Successfully deposited $%.2f\n", amount);  
        System.out.printf("New Balance: $%.2f\n", currentUser.getBalance());  
        System.out.println("-----\n");  
  
    } catch (NumberFormatException e) {  
        System.out.println(" ❌ Invalid amount. Please enter a valid number.\n");  
    }  
}
```

```
private static void withdrawMoney() {  
    System.out.println("\n----- WITHDRAWAL -----");  
    System.out.print("Enter amount to withdraw: $");  
  
    try {  
        double amount = Double.parseDouble(scanner.nextLine());  
        if (amount <= 0) {  
            System.out.println(" ❌ Amount must be greater than zero.\n");  
            return;  
        }  
  
        if (amount > currentUser.getBalance()) {  
            System.out.println(" ❌ Insufficient balance.\n");  
            return;  
        }  
  
        // ATM withdrawal limit (optional)  
        if (amount > 1000) {  
            System.out.println(" ❌ Withdrawal limit is $1000 per transaction.\n");  
            return;  
        }  
  
        currentUser.withdraw(amount);  
        System.out.printf(" ✅ Please take your cash: $%.2f\n", amount);  
        System.out.printf("New Balance: $%.2f\n", currentUser.getBalance());  
        System.out.println("-----\n");  
    }  
}
```

```
} catch (NumberFormatException e) {  
    System.out.println(" ❌ Invalid amount. Please enter a valid number.\n");  
}  
}
```

```
private static void transferMoney() {  
    System.out.println("\n----- TRANSFER -----");  
    System.out.print("Enter recipient account number: ");  
    String recipientAccountNumber = scanner.nextLine();  
  
    // Check if recipient exists and is not the same as sender  
    if (!accounts.containsKey(recipientAccountNumber)) {  
        System.out.println(" ❌ Recipient account not found.\n");  
        return;  
    }  
  
    if (recipientAccountNumber.equals(currentUser.getAccountNumber())) {  
        System.out.println(" ❌ Cannot transfer to your own account.\n");  
        return;  
    }  
  
    System.out.print("Enter amount to transfer: $");  
  
    try {  
        double amount = Double.parseDouble(scanner.nextLine());  
        if (amount <= 0) {  
            System.out.println(" ❌ Amount must be greater than zero.\n");  
            return;  
        }  
    }  
}
```

```
}
```

```
if (amount > currentUser.getBalance()) {  
    System.out.println(" ❌ Insufficient balance.\n");  
    return;  
}
```

```
UserAccount recipient = accounts.get(recipientAccountNumber);
```

```
// Perform transfer  
currentUser.withdraw(amount);  
recipient.deposit(amount);
```

```
System.out.printf(" ✅ Successfully transferred $%.2f to %s\n", amount,  
recipient.getHolderName());
```

```
System.out.printf("New Balance: $%.2f\n", currentUser.getBalance());  
System.out.println("-----\n");
```

```
} catch (NumberFormatException e) {  
    System.out.println(" ❌ Invalid amount. Please enter a valid number.\n");  
}  
}
```

```
private static void showTransactionHistory() {  
    System.out.println("\n----- TRANSACTION HISTORY -----");  
    System.out.println(currentUser.getTransactionHistory());  
    System.out.println("-----\n");  
}
```

```
private static void logout() {  
    currentUser = null;  
    System.out.println("\n ✅ Logged out successfully.\n");  
}  
}
```

```
class UserAccount {  
    private String accountNumber;  
    private String pin;  
    private String holderName;  
    private double balance;  
    private StringBuilder transactionHistory;  
  
    public UserAccount(String accountNumber, String pin, String holderName, double  
initialBalance) {  
        this.accountNumber = accountNumber;  
        this.pin = pin;  
        this.holderName = holderName;  
        this.balance = initialBalance;  
        this.transactionHistory = new StringBuilder();  
        transactionHistory.append("Account Created: $").append(String.format("%.2f",  
initialBalance)).append("\n");  
    }  
  
    public String getAccountNumber() {  
        return accountNumber;  
    }  
}
```

```
public String getHolderName() {  
    return holderName;  
}
```

```
public double getBalance() {  
    return balance;  
}
```

```
public boolean validatePin(String enteredPin) {  
    return this.pin.equals(enteredPin);  
}
```

```
public void deposit(double amount) {  
    this.balance += amount;  
    transactionHistory.append("Deposit: +$").append(String.format("%.2f", amount))  
        .append(" | Balance: $").append(String.format("%.2f", balance)).append("\n");  
}
```

```
public void withdraw(double amount) {  
    this.balance -= amount;  
    transactionHistory.append("Withdrawal: -$").append(String.format("%.2f", amount))  
        .append(" | Balance: $").append(String.format("%.2f", balance)).append("\n");  
}
```

```
public String getTransactionHistory() {  
    return transactionHistory.toString();  
}
```

}